

P3479R0

Enabling C pragma support in C++

Draft Proposal, 2024-10-15

This version:

<http://wg21.link/p3479r0>

Author:

[Joshua Cranmer](#)

Audience:

SG6, SG22, EWG

Project:

ISO/IEC 14882 Programming Languages — C++, ISO/IEC JTC1/SC22/WG21

Abstract

This is a proposal to extend C's floating-point pragmas for use in C++ code.

Table of Contents

- 1 Revision history**
- 2 Motivation**
 - 2.1 C pragmas
 - 2.2 C++ interactions
- 3 Proposal**
 - 3.1 Where pragmas may be applied.
- 4 Implementation experience**
- 5 Questions**

§ 1. Revision history

R0: first version

§ 2. Motivation

The C standard defines a number of pragmas to control the semantics of floating-point operations. C++ is a more complex language, and builds on C, which makes the extension of these pragmas non-trivial to extend into C++. The goal of this paper is to add into the standard extra clarity on how these pragmas interact with C++ features, without mandating their support or requiring deeper specification of floating-point behavior in C++.

At present, the C++ specification is almost entirely silent on the nature of issues that these pragmas bring up. [expr.const]/15 recommends, but does not requires, that implementations consistently implement constant expressions, and in [cfenv.syn], where the standard explicitly mentions that `FENV_ACCESS` support is optional. However, C++ is a more complex language than C, and it is not trivial to extend the rules in C to C++.

§ 2.1. C pragmas

C defines a number of floating-point pragmas that provide some means of local control over the floating-point model, without having to rely on compiler flags. These pragmas all have the same basic structure:

- If specified at file scope, they take effect until the end of the translation unit.
- If specified at the beginning of a compound statement (before any declarations or statements), they take effect until the end of the block.
- They take a parameter indicating the new state for their region of effect. For three of these, the parameter is a simple tri-state, `ON`, `OFF`, or `DEFAULT` (restoring the default state, which is usually implementation-defined and controlled by compiler command-line flags). The two rounding-mode pragmas specify a static rounding mode to use or a `FE_DYNAMIC` round option.

There are five such pragmas as of C23:

`CX_LIMITED_RANGE`

Allows complex multiplication, division, and absolute value to be computed according to their mathematical formulas, without regards to precision loss or special NaN handling rules.

FENV_ACCESS

If not present, the floating-point exception flags has unspecified value, and if the rounding mode is not default (except if set via `FENV_ROUND` and `FENV_DEC_ROUND`), behavior is undefined.

FENV_DEC_ROUND

FENV_ROUND

Allows the rounding mode for the block to be statically specified. These work correctly even if `FENV_ACCESS` is off. `FENV_ROUND` applies to binary floating-point types, and `FENV_DEC_ROUND` applies to decimal floating-point types.

FP_CONTRACT

Allows floating-point expressions to be contracted into a single expression, without intermediate rounding. The most common such contraction is contracting $a * b + c$ into a single `fma(a, b, c)`.

[TS 18661-5](#) provides an extension to C that adds some additional pragmas to control other optimizations than just contraction, including most notably allowing the assumption of the associativity law to rearrange floating-point expressions, and the ability to replace a division with a multiplication by a reciprocal (which might then be constant-folded, reused in common subexpressions, or hoisted out of a loop). Also, compilers may provide additional pragmas of their own. For example, [Clang provides a set of pragmas](#) for most of its floating-point model flags. MSVC also provides a smaller set of pragmas, although not the standard C pragmas themselves.

These pragmas essentially allow C users to specify the floating-point model of their code on a lexically-scoped basis.

§ 2.2. C++ interactions

C++ adds several more declaration kinds that can contain functions than C does. This makes the effect of a pragma that exists outside of function definitions somewhat unclear. For example, if such a pragma occurs inside of a class scope, does it apply to all subsequent functions declared in that class but not those outside of the class? Does a subsequent pragma at the global scope override that pragma definition? Similar questions can be asked for other declarations that can contain declarations, like namespaces, external blocks, and export declarations. The easiest way to handle these scenarios is to simply ban the use of the pragma in any of these new contexts, and only allow it in places where its interpretation would be unambiguous.

Another major concern is that C++ creates more opportunities for generic code than C does. In particular, consider code like this:

```
class smart_float {
    friend smart_float operator+(smart_float, smart_float);
    friend smart_float operator*(smart_float, smart_float);
};

void example(smart_float a, smart_float b, smart_float c) {
    {
        #pragma STDC FP_CONTRACT ON
        // This should generate an fma operation...
        a * b + c;
    }
    {
        #pragma STDC FP_CONTRACT OFF
        // ...but this shouldn't
        a * b + c;
    }
}
```

Pragmas are not sufficient to allow one to write generic code in this fashion in C++: extra functionality would be required to define a function which will have multiple implementations depending on the currently applied floating-point pragma state. This could potentially be achieved with an attribute on the function that would allow all relevant pragma state to be inherited.

§ 3. Proposal

This paper does not intend to propose any requirements that C++ implementations must support these pragmas, nor does it intend to any pragmas that are not already in C23.

§ 3.1. Where pragmas may be applied.

The C rules on where the various pragmas may be applied are as follows:

The pragma shall occur either outside external declarations or before all explicit declarations and statements inside a compound statement.

The nearest adaption of these rules to C++ is to require that the pragma shall occur either where a declaration would belong to the global scope or before all explicit declarations and statements inside a compound statement. Such a rule would provide for these cases:

```
namespace A {
    #pragma STDC FENV_ACCESS ON // ill-formed
}
class B {
    #pragma STDC FENV_ACCESS ON // ill-formed
};
extern "C" {
    #pragma STDC FENV_ACCESS ON // not ill-formed, continues to end of the f:
}
void d() {
    #pragma STDC FENV_ACCESS ON // not ill-formed, continues to end of funct:
    if (0.1 + 0.2 == 0.3) {
        #pragma STDC FENV_ACCESS OFF // not ill-formed
    }
}
template <
#pragma STDC FENV_ACCESS ON // ill-formed
float f = 0.1 + 0.2>
void e(
#pragma STDC FENV_ACCESS ON // ill-formed
float g = f);
```

§ 4. Implementation experience

At present, the clang compiler already implements support for the C pragmas that it supports in C++. It does not yet support the two rounding mode pragmas added in C23, but the other ones, as well as a few custom pragmas for other properties of floating-point operations (primarily but not exclusively fast-math-related properties). These pragmas are prohibited in the class scope, and generally if they occur in the middle of a declaration (e.g., in the middle of a function argument list), but they are not prohibited within a namespace scope.

MSVC does not implement any of the C pragmas yet, but they do implement a set of similar pragmas. MSVC's custom pragmas can only be applied at global or namespace scope, and cannot be applied anywhere within a function boundary (unlike the C pragmas).

None of these implementations have any attribute or similar feature to make a function implementation aware of the pragma state of the caller.

§ 5. Questions

- Do people want to see support for the C floating-point pragmas in C++?
- Do people want to see work continue on an attribute to make function implementations aware of the pragma state of their caller?